Software Production Engineering MiniProject

Scientific Calculator

Anuja Jose MT2025022

DOCKERHUB: https://hub.docker.com/repository/docker/anujajose/calculator/general
GITHUB:https://github.com/anuja-jmk/pipelined-miniproject

March 7, 2026

CSE 816 Software Production Engineering

Prof. B. Thangaraju

International Institute of Information Technology
Bangalore

Contents

Table of Contents i

1 Objective 1

2 Tools Used 2

3 Stages 3

3.1 Development - Python . . . 3

3.2 Testing - Pytest . . . 3

3.3 Containerization - Docker . . . 4

3.4 Configuration - Ansible . . . 5

3.4.1 Test Ansible . . . 6

3.5 Configuring credentials . . . 6

3.6 Pipeline - Jenkins . . . 7

3.7 Outputs . . . 8

Chapter 1

Objective

To get familiarized with the Devops stages and steps, to build a pipeline that tracks the stages from developing to testing till deployment. By building a mini application of a scientific calculator, design and implement it in a CI/CD pipelined system.

Chapter 2

Tools Used

- Github
  Git is used for version control and for distributed internet based version control github is used for viewing the commits and version histories.

- Jenkins
  For the automation of the stages of the software, building, testing and deployment.

- Docker
  Docker is used for the containerization of the complete application for portable behavior of the software.

- Ansible
  Ansible configuration management tool is used for configuring and executing the tasks systematically on the host system

Chapter 3

Stages

3.1 Development - Python

Code development is the initial step that involves going back to versions back and forth. The scientific calculator program was written in python in vs code and also using a simple ui framework, Dash.

The environment for the python code development was made and requirements.txt was also written for reproducing the python environment. Environment is created and activated using the commands in the commandline:
python -m venv .venv
source ./.venv/bin/activate
For running the application using gunicorn: **gunicorn app:server -b 0.0.0.0:8050**

```
root@85553f95a80b:/app# gunicorn app:server -b 0.0.0.0:8050
[2026-03-07 07:29:29 +0000] [7] [INFO] Starting gunicorn 25.1.0
[2026-03-07 07:29:29 +0000] [7] [INFO] Listening at: http://0.0.0.0:8050 (7)
[2026-03-07 07:29:29 +0000] [7] [INFO] Using worker: sync
[2026-03-07 07:29:29 +0000] [7] [INFO] Control socket listening at /app/gunicorn.ctl
[2026-03-07 07:29:29 +0000] [9] [INFO] Booting worker with pid: 9
```

A Repository is created in the github account and it is cloned and then for each changes, files are committed and pushed to the repo for the version control.

3.2 Testing - Pytest

Run the unit tests using the pytest library.

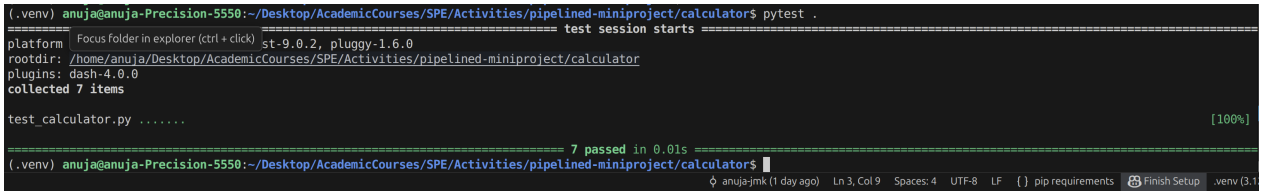

Figure 3.1: Running unit tests with pytest

3.3 Containerization - Docker

Containerization of the application is done using Docker. Containerization of the application is necessary to avoid dependency based conflicts and to deploy the system in the same environment as the development environment.

Initially dockerfile is created to build the image.

```
Dockerfile > ...
FROM python:3.10-slim

WORKDIR /app

COPY ./calculator /app

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 8050

CMD ["gunicorn", "app:server", "-b", "0.0.0.0:8050"]
```

Figure 3.2: Dockerfile

Steps:

- Base image selection
- Declare the working directory
- Copy the contents from calculator folder to the docker image
- Install the requirements.txt using pip
- Expose the port of the application

- Run the server

Build the image using:
docker build -t calculator .

Run the container using the command:
docker run -d -p 7777:8050 –name ctr1 calculator

Visit http://localhost:7777 to check if the application is accessible.

3.4 Configuration - Ansible

Configuration of the server to run is done using Ansible. Server used for this system is localhost hence server is local

Playbook contains the specific set of task for each of the servers which will be executed sequentially. This will ensure that the st of instructions are executed in the exact same sequence each time so that it will be an automated system

```
deploy.yml
---
- name: Pull Docker Image from Docker Hub
  hosts: localhost
  remote_user: anuja
  become: false
  tasks:
    - name: Pull Docker Image
      docker_image:
        name: "anujajose/calcu"
        source: pull
      register: docker_pull_result

    - name: Display Docker Pull Result
      debug:
        var: docker_pull_result

    - name: Start Docker service
      service:
        name: docker
        state: started
    - name: Running container
      shell: docker run -d -p 7777:8050 --name calculator anujajose/calcu
```

Figure 3.3: Ansible playbook file

Playbook:

- hosts : sets the host server name which was defined in the inventory.ini

- remoteuser: username , become = false, to run as a non-root user
- tasks: are the set of instructions that are to be executed on the server.
- task1: to pull the docker image from the docker registry.
- task2: display the result of the docker pull, to see if it failed or succeeded.
- task3: to run the docker container on the given server.

3.4.1 Test Ansible

Use the command : **ansible-playbook -i inventory deploy.yml**

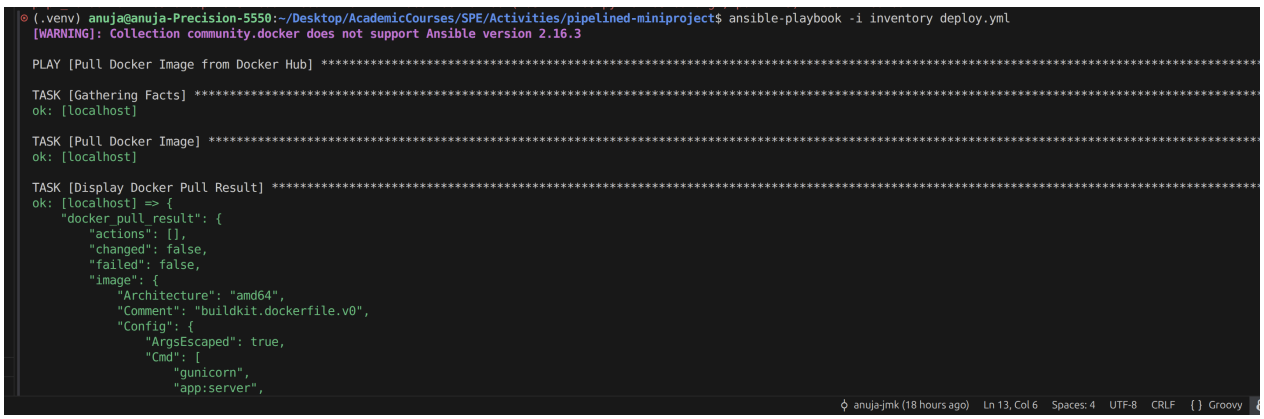

Figure 3.4: Running the container using ansible playbook

3.5 Configuring credentials

For Setting a Webhook we need to setup credentials for github. Ngrok has to be installed and dockerhub and github credentials has to be updated.

Steps in Ngrok

Ngrok is an API gateway, which can convert a local API to public API as used in this project. Its necessary so that the public url of jenkins is available to provide the API for the github webhook.

Run
ngrok http 8080

```
ngrok

 One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status                online
Account                       ajmk (Plan: Free)
Version                       3.35.0
Region                        United States (us)
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://bosomed-nitrosylsulfuric-itzel.ngrok-free.dev -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              0       0       0.00    0.00    0.00    0.00
```

Figure 3.5: Ngrok port forwarding

Copy the URL Forwarded for later use.

3.6 Pipeline - Jenkins

Jenkins is an open source tool mainly used for automation and setting up of pipelines. Initially Jenkins is installed and Configuration of credentials is done. New Pipeline project is created and the pipeline script is added in the pipeline section. Trigger is enabled for github with poll SCM with webhook and build is taken from the repo.

Given here is the complete pipeline script overview:

pipeline
agent any
triggers
githubPush()
TRIGGER SETTING FOR TRIGGERING THE PIPELINE

environment
SETTING ENVIRONMENT VARIABLES

stages:
THESE ARE THE MAIN STAGES IN THE PIPELINE
stage('Checkout')
CHECKOUT THE BRANCH AND PULL THE CHANGES FROM THE GITHUB REPO

stage('Cleanup Old Containers')
CLEANUP OR REMOVE THE OLD CONTAINER IF IT EXIST

stage('Build Docker Image')
BUILD THE DOCKER IMAGE USING THE NEW DOCKER FILE
stage('Run Unit Tests in Test Container')
RUN THE UNIT TESTS WITH A CONTAINER

stage('Remove Test Container')

REMOVE THE TEST CONTAINER

stage('Push Docker Images')
PUSH THE DOCKER IMAGES TO THE DOCKER HUB

stage('Run Ansible Playbook')
RUN THE ANSIBLE PLAYBOOK TO DEPLOY THE CONFIGURED SERVER
post
success
IF THE PIPELINE IS A SUCCESS SENT MAIL

failure **mail to:**

failure
IF THE PIPELINE FAILED AT SOMEPOINT SENT MAIL mail to:

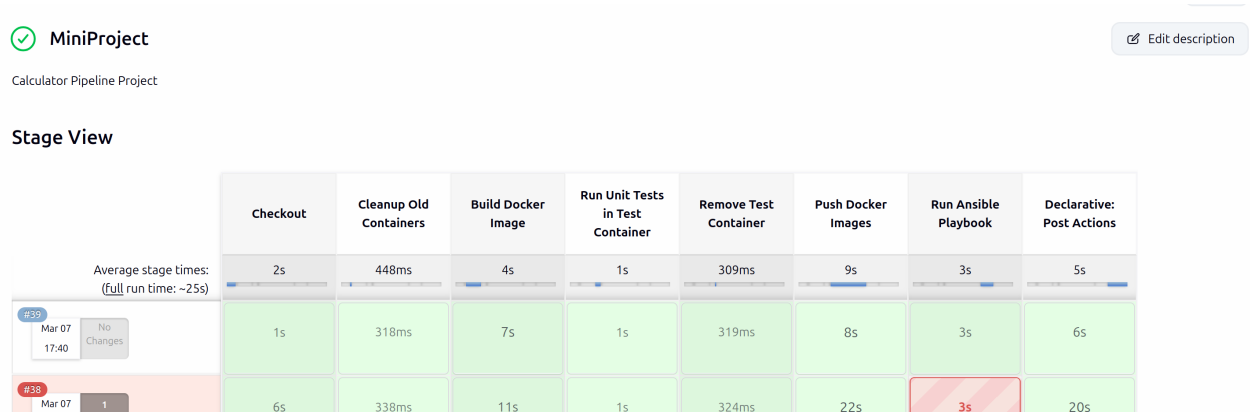

Figure 3.6: STAGES IN THE PIPELINE

3.7 Outputs

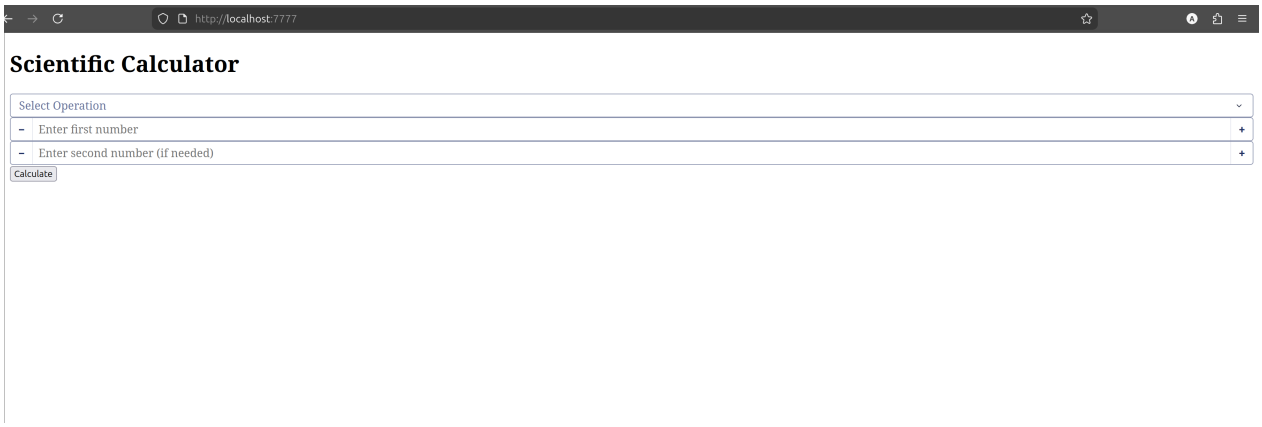

Figure 3.7: UI of the scientific calculator

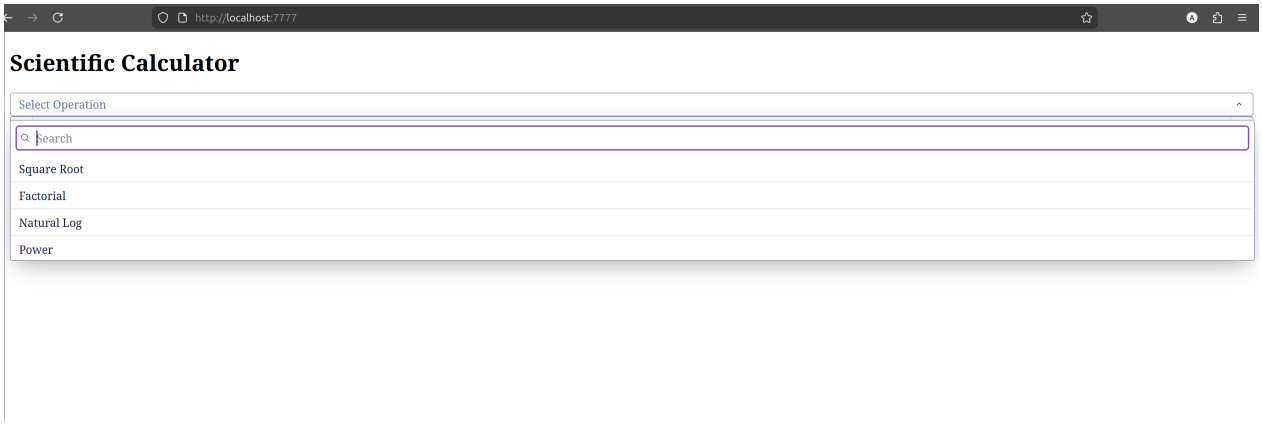

Figure 3.8: Menu of the Calculator

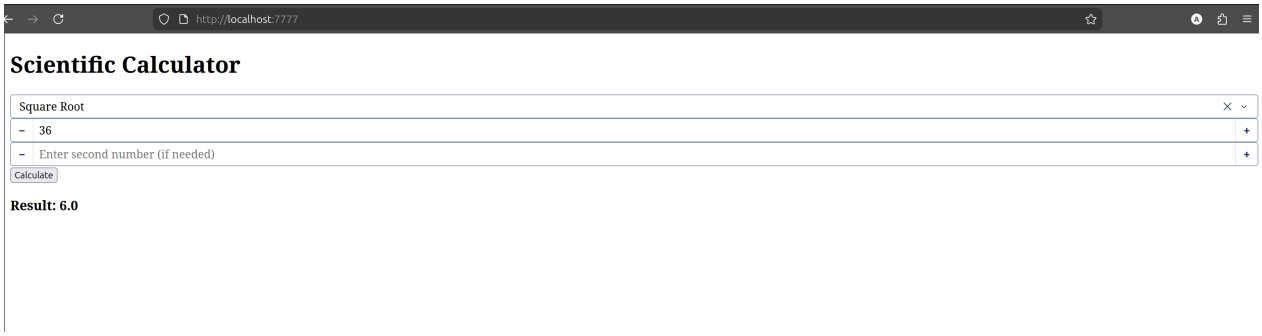

Figure 3.9: Square Root Operation

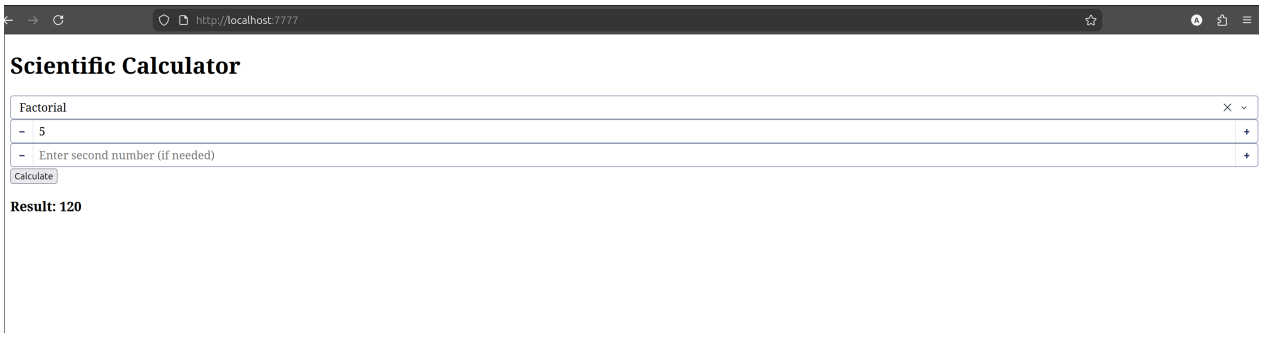

Figure 3.10: Factorial Operation

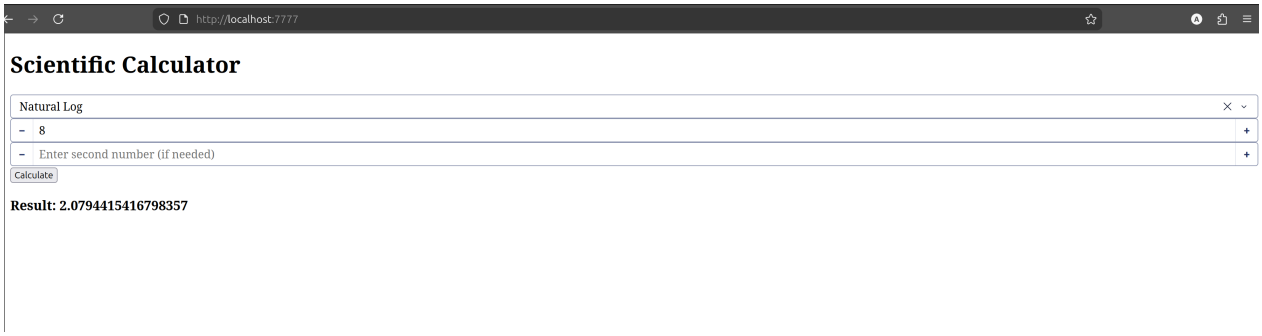

Figure 3.11: Natural Logarithm Operation

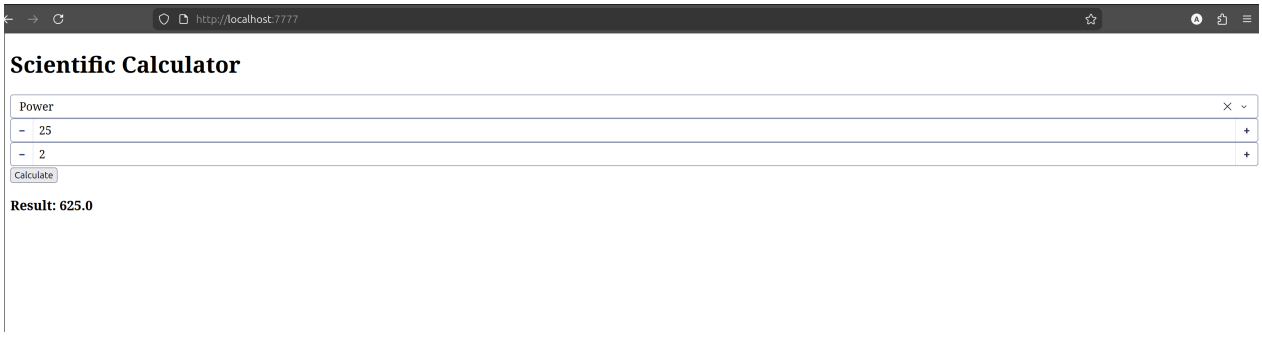

Figure 3.12: Power Operation